



Universidad Nacional Autónoma de México
Facultad de Ciencias

MODELADO Y PROGRAMACIÓN

Canek Peláez Valdés
Miriam Torres Bucio
Cynthia Lizbeth Sánchez Urbano

PROYECTO 1

SISTEMA CLIENTE-SERVIDOR EN C++

GRUPO 7079
SÁNCHEZ GARCÍA HUGO

Resumen

Se elaboró un sistema cliente-servidor en C++ para mensajes de texto, con el protocolo TCP para que varios clientes pudieran comunicarse entre sí por medio de un servidor central, que administra a los usuarios conectados, si están disponibles y las salas de conversación. Los clientes pueden enviarse y recibir mensajes por línea de comandos.

La comunicación entre servidor y clientes es por JSON delimitados por saltos de línea, para formatear los datos de identificación de los clientes, la lista de los que están conectados, sus cambios de estado, los mensajes privados que se envían y los mensajes públicos de las salas, y creación de salas, invitaciones y desconexiones.

Luego, como el servidor debe atender a la vez a varios clientes, se usa un hilo independiente por cada conexión, y las estructuras de datos compartidas se protegen por "mutex" de la biblioteca estándar, en especial para evitar el problema de carrera.

Todo el sistema, tanto el del servidor como el del cliente están en C++17 con sockets de Berkeley/POSIX, la biblioteca estándar de hilos y la de encabezado nlohmann/json para los JSON, y el sistema de construcción es CMake porque resultó el más sencillo para compilar. En las pruebas se validaron la identificación, manejo de usuarios, comunicación entre clientes, creación de salas, invitaciones, entrada y salida de salas, y tratar mensajes inválidos y el límite máximo de tamaño del mensaje, que es de 1 048 576 bytes.

1. Introducción

Como habíamos visto en clase, como dos procesos que se ejecutan en máquinas distintas no pueden compartir directamente su memoria, requieren de un mecanismo de comunicación para que se compartan información estructuradamente.

Y precisamente una de las alternativas más usadas es comunicar las máquinas por sockets, y un socket puede entenderse como un punto final de comunicación que deja enviar y recibir información en una red. En el caso de TCP, la comunicación se da por una conexión a flujo, que entrega ordenadamente la información de los bytes que se transmiten.

Este proyecto consiste en implementar un servidor de chat que pueda atender a varios clientes concurrentemente. El servidor funciona como punto central para mantener el estado de los usuarios conectados, las salas existentes y determinar cuáles clientes deben recibir cada mensaje.

El sistema debe interpretar varias operaciones y mantener la información mientras las

conexiones siguen activas. Sea por caso que para enviar un mensaje privado es necesario localizar al destinatario y si no existe no se le envía nada, o para enviar un mensaje a una sala es necesario verificar antes que el emisor sea miembro, y para aceptar una invitación es necesario validar que la invitación exista.

Entonces el sistema se comunica por sockets TCP, representa los mensajes con JSON y usa estructuras de datos concurrentes para los usuarios y las salas.

2. Diseño

2.1. Arquitectura

Como ya se dijo, la arquitectura es cliente-servidor. Hay un proceso servidor que permanece escuchando conexiones entrantes en un puerto TCP específico (que se puede cambiar), y cada cliente establece una conexión el servidor para enviar operaciones y recibir respuestas o notificaciones del estado de los otros usuarios.

Entonces la arquitectura general es de este estilo:

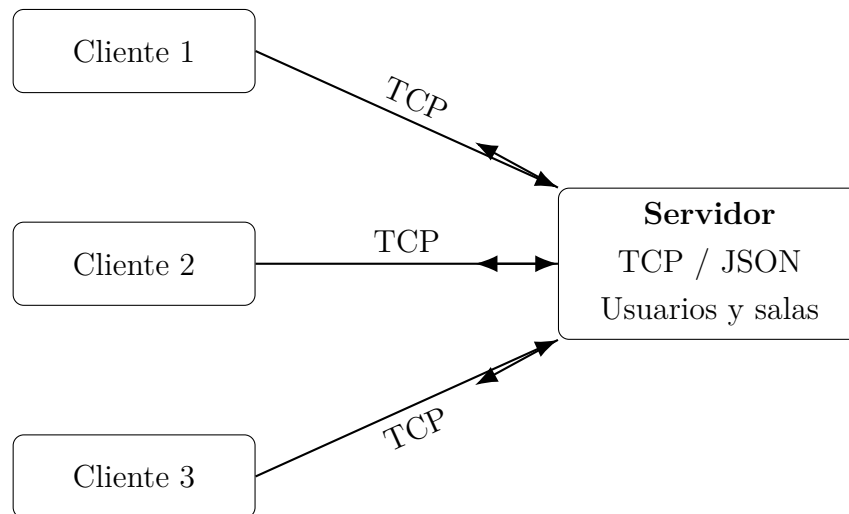


Figura 1: Arquitectura general

El servidor es el encargado de mantener el estado global de todo el sistema. Entonces los clientes no mantienen directamente información sobre los demás usuarios ni sobre la pertenencia a las salas, y en lugar de eso, hacen solicitudes al servidor y reciben información de los cambios.

2.2. Modelo de comunicación

La comunicación usa JSON que terminan en salto de línea `\n`, así aunque hay un flujo continuo de bytes por TCP, el servidor tiene un límite entre mensajes.

Por ejemplo, una solicitud de identificación se escribe como:

```
{"type":"IDENTIFY","username":"Hugo"}
```

El servidor procesa los bytes que recibe hasta encontrar `\n`, luego interpreta la cadena como JSON y obtiene el campo de `type`, que indica la operación solicitada.

Entonces esto también deja que un mensaje sea recibido en varios fragmentos, porque la aplicación no depende de que una llamada a `read()` reciba un mensaje completo.

2.3. Modelo de datos

El estado de los usuarios se representa con la estructura de `InfoCliente`:

```
1 struct InfoCliente {  
2     int conexion_cliente;  
3     std::string username;  
4     std::string status = "ACTIVE";  
5 };
```

Cada usuario tiene un socket asociado, un nombre de usuario y un estado.

Y los únicos estados que hay son:

- ACTIVE;
- AWAY;
- BUSY;

Los usuarios conectados se almacenan en un `std::unordered_map`, con su nombre de usuario como llave. Se eligió así para localizar a los usuarios por nombre con menor complejidad en tiempo $O(1)$ en el caso promedio.

Las salas utilizan la estructura de:

```
1 struct InfoSala {  
2     std::unordered_set<std::string> miembros;  
3     std::unordered_set<std::string> invitados;  
4 };
```

Cada sala tiene dos conjuntos ajenos, donde el primero es de los usuarios que ya pertenecen a la sala y el segundo es de los que recibieron invitación pero todavía no se unen.

Las salas se almacenan también por un `std::unordered_map` con su nombre como llave.

2.4. Operaciones

Estas son las únicas operaciones que el cliente puede solicitar al servidor:

Operación	Descripción
IDENTIFY	Identifica al cliente por nombre de usuario.
STATUS	Cambia el estado del usuario entre ACTIVE , AWAY y BUSY .
USERS	Solicita la lista de usuarios conectados y sus estados.
TEXT	Envía un mensaje privado a otro usuario.
PUBLIC_TEXT	Envía un mensaje a todos los usuarios conectados menos al emisor.
NEW_ROOM	Crea una nueva sala y hace al creador su primer miembro.
INVITE	Invita a uno o más usuarios a una sala.
JOIN_ROOM	Para que un usuario invitado se una a una sala.
ROOM_USERS	Obtiene la lista de miembros de una sala.
ROOM_TEXT	Envía un mensaje a los demás miembros de una sala.
LEAVE_ROOM	Para que un usuario abandone una sala.
DISCONNECT	Termina voluntariamente la conexión del usuario.

Cuadro 1: Lista de operaciones del servidor.

2.5. Identificación de usuarios

La identificación es estrictamente el primer paso para que una conexión al servidor sea válida. La única operación que el servidor acepta mientras el usuario no se haya identificado es **IDENTIFY**. Cualquier otra entrada que se le pase da respuesta de error y cierra la conexión.

Si el nombre solicitado no está registrado todavía, el servidor asocia el socket con el usuario y pone su estado inicial como **ACTIVE**. Además, cuando un usuario se registra y el servidor ya tenía usuarios previos, los demás usuarios conectados reciben una notificación de **NEW_USER**.

El límite para el nombre de usuario es de ocho caracteres.

2.6. Modelo de salas

La administración de salas es por invitación y aceptación.

O sea que primero, un usuario crea una sala por la operación de `NEW_ROOM`, y ya que la crea se incorpora en automático como miembro.

Después, un miembro puede invitar a otros usuarios mediante con `INVITE`, pero esa invitación no convierte al usuario invitado en miembro, solo lo agrega al conjunto de invitados.

El usuario invitado puede usar `JOIN_ROOM` para unirse, y al aceptar se elimina del conjunto de invitados y se agrega al de miembros.

Por último, un miembro puede usar `LEAVE_ROOM` para abandonar la sala, y cuando una sala se queda sin miembros se elimina del servidor.

2.7. Concurrency

El servidor debe atender en simultáneo varias conexiones y para eso cuando se acepta una nueva conexión también se crea un hilo independiente así:

```
1 std::thread(manejar_cliente, socket_cliente).detach();
```

La función de `manejar_cliente` contiene el ciclo principal de procesamiento de cada cliente.

El uso de varios hilos da precisamente el acceso concurrente a las estructuras que ya se dijeron. Por ejemplo, mientras que un hilo modifica el estado de un usuario, otro podría estar solicitando la lista de usuarios.

Y justamente para proteger los datos se usan dos mutex:

```
1 std::mutex mutex_clientes;  
2 std::mutex mutex_salas;
```

El primero protege el mapa de usuarios y el segundo las estructuras de las salas.

Entonces se pueden ejecutar distintas operaciones concurrentemente siempre que no necesiten modificar el mismo estado compartido.

3. Implementación

3.1. Lo que se usó

Para el proyecto se usó lo siguiente:

- C++ 17 para el servidor y para el cliente, y la y la Standard Library para las estructuras, hilos y cadenas.
- Sockets POSIX/Berkeley para la comunicación TCP.
- TCP para comunicarse en la misma red.
- nlohmann/json para analizar los JSON.
- CMake para configurar y construir todo el proyecto.

El proyecto se divide en dos ejecutables que son `server.cpp` para el proceso del servidor, y `client` para el del cliente.

La biblioteca `json.hpp` está en el directorio `external`, así que el proyecto puede compilarse sin tener que instalarla.

3.2. Inicialización del servidor

La primera etapa del servidor es crear un socket:

```
1 int socket_servidor = socket(AF_INET, SOCK_STREAM, 0);
```

Para lo que se usará la familia de direcciones `AF_INET`, `SOCK_STREAM` para un socket de flujo y el protocolo predeterminado de TCP.

Luego se configura la dirección con:

```
1 direccion_servidor.sin_family = AF_INET;  
2 direccion_servidor.sin_addr.s_addr = INADDR_ANY;  
3 direccion_servidor.sin_port = htons(8080);
```

El servidor se asocia al puerto por `bind()` y empieza a aceptar conexiones por `listen()`.

Y por último, el servidor entra en un ciclo en donde llama a `accept()`.

3.3. Atención de conexiones

Cada conexión aceptada obtiene una conexión de cliente independiente, o sea un descriptor de socket, y el servidor crea un hilo para atenderlo:

```
1 if(socket_cliente >= 0) {  
2     std::thread(manejar_cliente, socket_cliente).detach();  
3 }
```

Se usa `detach()` para dejar que el hilo quede ejecutándose independientemente del hilo principal, mientras que el principal continúa aceptando nuevas conexiones.

En `manejar_cliente` se mantiene un ciclo que recibe mensajes, los interpreta como JSON y ejecuta la operación que se le pide.

3.4. Recepción de mensajes

La función para reconstruir los mensajes es `leer_linea`, y usa el salto de línea `\n` como delimitador.

Y como ya se había explicado, la función no supone que una llamada a `read()` sea un mensaje completo, así que podría obtener los bytes progresivamente y acumularlos en un `std::string` hasta encontrar salto de línea.

Además, se estableció un límite máximo de:

$$1024^2 = 1\,048\,576 \text{ bytes.}$$

Y si el mensaje supera dicho tamaño, el servidor responde con una operación inválida y termina la conexión.

Es claro que esto es para evitar que un cliente pueda provocar un crecimiento ilimitado de la cadena utilizada para almacenar un mensaje recibido.

3.5. Procesamiento de los JSON

Una vez que se reconstruye una línea, el servidor usa:

```
1 json mensaje = json::parse(cadena_buffer);
```

para convertir el texto en JSON.

El tipo de operación solicitada se obtiene con:

```
1 std::string tipo = mensaje.at("type");
```

Aquí es donde se selecciona la rama correspondiente por una condicional.

Si el mensaje no puede analizarse como JSON, tiene campos faltantes o un valor inválido, se captura la excepción y se responde con:

```
{  
    "type": "RESPONSE",
```

```
"operation": "INVALID",  
"result": "INVALID"  
}
```

En seguida se termina la conexión.

3.6. Envío de mensajes

Todos los mensajes enviados por el servidor se serializan usando `dump()` y se les agrega un salto de línea así:

```
1 std::string datos = mensaje.dump() + "\n";  
2 write(conexion_cliente, datos.c_str(), datos.length());
```

La función de `enviar_mensaje` centraliza esto comportamiento y evita que cada operación deba serarse una por una.

Por ejemplo, cuando se envía un mensaje privado, el servidor construye:

```
{  
    "type": "TEXT_FROM",  
    "username": "emisor",  
    "text": "Hola"  
}
```

y lo transmite al socket del destinatario.

3.7. Mensajes privados y públicos

Para `TEXT`, primero el servidor busca al destinatario en el mapa de clientes, y si existe envía el mensaje únicamente a ese socket.

Si no, da una respuesta indicando que el usuario solicitado no está disponible.

Por otro lado, `PUBLIC_TEXT` recorre todos los usuarios conectados y a todos envía el menos a quien lo mandó.

3.8. Administración de salas

Con `NEW_ROOM` se verifica que el nombre de la sala no exista. Cuando una se crea, se inserta en automático al usuario creador en el conjunto de miembros.

Para limitar el nombre de las salas menor o igual a 16 caracteres se implementó una función auxiliar:

```
1 bool nombre_sala_valido(const std::string& nombre_sala) {  
2     return nombre_sala.length() <= 16;  
3 }
```

Y se usa en todas las operaciones de salas.

Con `INVITE` se comprueba que la sala exista y que el emisor sea miembro de ella. Después comprueba los usuarios destinatarios y envía las invitaciones.

En `JOIN_ROOM`, se comprueba que la sala exista y que el usuario tenga una invitación pendiente. Si ambas condiciones se cumplen, elimina la invitación y agrega al usuario a los miembros.

`ROOM_USERS` y `ROOM_TEXT` requieren que el usuario solicitante pertenezca a la sala. Así se impide que un usuario no miembro pueda consultar sus integrantes o usarla para transmitir mensajes.

Y por último, `LEAVE_ROOM` elimina al usuario de los miembros y notifica a los integrantes que quedan.

3.9. Desconexión

La desconexión puede ocurrir explícitamente al usar `DISCONNECT` o como consecuencia de un error.

Cuando un usuario identificado abandona el servidor, se dan dos tareas. Primero, se elimina al usuario de las salas en las que participaba y se notifica a los demás miembros por mensajes con `LEFT_ROOM`. Después se elimina su registro del mapa de clientes y se notifica a los usuarios restantes mediante un mensaje `DISCONNECTED`.

Y así ya no quedan referencias cuando un usuario sale.

4. Pruebas

La implementación se sometió a pruebas manuales utilizando varias conexiones de cliente. Se quería ver el funcionamiento normal como el comportamiento a entradas inválidas.

4.1. Identificación

Se probó IDENTIFY con nombres válidos y con otros que superaban el límite de 8 caracteres. Uno de exactamente ocho caracteres fue aceptado correctamente, y uno de mayor longitud se rechazó y terminó la conexión.

También se probó la identificación duplicada. Cuando un segundo cliente intentó utilizar un nombre ya registrado, el servidor lo detectó por el mapa de clientes.

4.2. Estados de usuario

Se verificaron ACTIVE,

AWAY y

BUSY.

Se probó un valor fuera del conjunto permitido. El servidor lo identificó como inválido y terminó la conexión.

4.3. Comunicación

Se probaron mensajes privados y públicos usando más de un cliente.

Para mensajes privados se probó que solamente el destinatario recibiera la notificación de TEXT_FROM. Para los públicos se verificó que los demás usuarios recibieran PUBLIC_TEXT_FROM.

También se probó que un destinatario inexistente produjera la respuesta correspondiente.

4.4. Salas

Las operaciones para creación, invitación, incorporación, consulta, envío de mensajes y abandono de salas fueron probadas individualmente.

Se verificaron en especial estos casos:

- creación de una sala nueva;
- intento de crear una sala existente;
- invitación de usuarios;
- intento de invitar usuarios inexistentes;
- incorporación de un usuario invitado;

- intento de incorporación sin invitación;
- consulta de integrantes;
- envío de mensajes a una sala;
- abandono de una sala.

También se verificó el límite de 16 caracteres para los nombres de las salas. Una de exactamente 16 caracteres fue aceptada, y una con 17 fue rechazada.

4.5. Mensajes inválidos

Se probó el envío de operaciones inexistentes, como por ejemplo:

```
{"type": "HOLA"}
```

El servidor respondió como:

```
{  
  "type": "RESPONSE",  
  "operation": "INVALID",  
  "result": "INVALID"  
}
```

y después terminó la conexión.

También se probó enviar una operación sin un campo obligatorio, y en este caso el acceso por `at()` genera una excepción, que es capturada y se procesa como mensaje inválido.

4.6. Tamaño máximo de mensaje

Una prueba adicional consistió en enviar un mensaje cuyo tamaño superaba 1 048 576 bytes.

El servidor detectó que el mensaje superó el límite y respondió con:

```
{  
  "type": "RESPONSE",  
  "operation": "INVALID",  
  "result": "INVALID"  
}
```

De inmediato cerró la conexión correspondiente.

Con eso también se comprobó que el límite de tamaño se aplica al mensaje completo en vez de al individual de cada llamada a `read()`.

5. Conclusiones

El proyecto permitió hacer un sistema de comunicación cliente-servidor con varias conexiones a la vez con TCP. El servidor tiene un estado global compuesto por usuarios y salas, y cada cliente se comunica con el servidor por JSON.

En el desarrollo se comprendió que TCP da un flujo de bytes y no mensajes individuales. Para eso se usó un delimitador de línea y una cadena acumuladora para reconstruir mensajes aunque sean recibidos en varios fragmentos.

El uso de JSON permitió representar uniformemente las operaciones. Con el campo de `type` se permitió distinguir las diferencias de las solicitudes, y los campos adicionales con los datos necesarios por operación.

Para las salas se requirió diferenciar específicamente entre miembros e invitados para poder representar las operaciones de creación, invitación, aceptación, comunicación y abandono.

Para la concurrencia, el servidor crea un hilo por cliente, que deja que una conexión quede esperando los datos sin impedir que el servidor pueda atender a otras conexiones, y mutex se usó para poder proteger las estructuras compartidas entre los hilos.

En conclusión, se logró implementar el proyecto en C++ con sockets, seriación de información y concurrencia,

Referencias

- [1] cppreference.com. *C++ Reference*. Documentación de referencia.
- [2] The Open Group. *The Open Group Base Specifications*.
- [3] Linux man-pages. *socket(2) – create an endpoint for communication*.
- [4] Linux man-pages. *bind(2) – bind a name to a socket*.
- [5] Linux man-pages. *listen(2) – listen for connections on a socket*.
- [6] Linux man-pages. *accept(2) – accept a connection on a socket*.
- [7] Linux man-pages. *read(2) – read from a file descriptor*.
- [8] Linux man-pages. *write(2) – write to a file descriptor*.

[9] Niels Lohmann. *JSON for Modern C++*.

[10] Kitware. *CMake Documentation*.